

NAS-in-the-Loop: Trajectory-Driven Neural Architecture Search for Safe Autonomous CPS

Zayah Cortright[†], Prateek Ganguli[†], Tingan Zhu[†], Samarjit Chakraborty^{†‡}

[†]The University of North Carolina at Chapel Hill, USA

[‡]TUM Institute for Advanced Study, Germany

{zayah, pganguli, tingan, samarjit}@cs.unc.edu

Abstract—Neural Architecture Search (NAS) is commonly formulated as a supervised learning optimization problem where architectures are ranked using prediction accuracy. However, in cyber-physical systems, perception models operate with closed-loop controllers, and small estimation errors can propagate into unsafe system behaviors. This paper therefore proposes a closed-loop trajectory-level NAS framework for autonomous systems using the FITENTH platform. Instead of optimizing prediction accuracy, the proposed method calls perception modules directly in simulation and calculates cross-track error as the NAS objective. Experimental results on multiple racing tracks show the trajectory-driven NAS selects modules that reduce normalized composite safety-cost relative to accuracy-driven NAS and handcrafted baselines. The largest individual improvement is a 36.8% reduction in cross-track error relative to the handcrafted baseline mean, representing an additional 13.2% reduction compared to the accuracy-driven NAS. The results support the central premise that neural architectures for autonomous systems should be selected using closed-loop behavioral evaluation rather than standalone prediction accuracy.

Index Terms—neural architecture search, cyber-physical systems, Bayesian optimization, safety metrics, neural networks

I. INTRODUCTION AND RELATED WORK

Autonomous cyber-physical systems (CPS) often rely on learned perception models to estimate the state variables used by their controllers. Errors in these estimates directly affect control decisions, and their impact on closed-loop behavior depends on the controller structure, track geometry, and vehicle state at each timestep. A perception model that achieves low validation loss may still produce estimation patterns that cause the controller to oscillate, drift from the desired path, or reduce wall clearance during operation. Conversely, a model with slightly higher prediction error might produce estimation residuals that the controller absorbs gracefully, yielding a safer overall trajectory. Existing works on *neural architecture search* (NAS) systematically ignore this distinction by selecting neural architectures based on prediction accuracy alone.

This work studies the problem in the FITENTH autonomous racing platform [1], a well-established testbed for embedded autonomous control. At each control step, a 1080-beam LiDAR scan is processed by three independent neural networks that estimate left-wall distance, track width, and heading error. These estimates are fed to a fixed racing controller that computes steering and speed commands. Because all three quantities enter the controller simultaneously, estimation errors

in any of them can propagate into cross-track deviation, heading drift, steering oscillation, or reduced wall clearance. The question this work addresses is which neural architectures for these three estimation tasks produce the safest closed-loop behavior when embedded in the controller, rather than which architectures minimize held-out prediction loss.

Related work on NAS: Neural Architecture Search (NAS) automates network design by searching a predefined architecture space according to a target objective [2]. Early methods relied on reinforcement learning [3] or evolutionary search [4]; differentiable NAS reduces cost by relaxing discrete choices into continuous parameters updated by gradient descent [5], [6]. Hardware-aware NAS extends the objective to include latency, memory, and energy [7], [8]. All of these formulations, however, evaluate architectures as standalone predictors or under static deployment constraints. For CPS, where perception outputs are intermediate quantities consumed by a feedback controller, this is insufficient: two architectures with nearly identical prediction accuracy can produce substantially different closed-loop trajectories depending on how their specific error patterns interact with the downstream control law.

Related work on safety-informed NAS for CPS: A body of recent work has studied how perception error and implementation timing uncertainty affect CPS safety through formal verification [9], scheduling and timing analysis [10], [11], and reachability-based quantification of state-set expansion under perception uncertainty [12]. These results establish that component-level metrics do not fully characterize closed-loop system behavior, a point made directly in [13] in the context of perception evaluation for autonomous systems. An emerging direction integrates system-level safety signals directly in the architecture search loop, using reachability-based metrics to guide differentiable NAS for CPS perception [14]. The present work pursues a complementary approach suited to a simulation-driven setting: rather than computing formal safety metrics during search, the NAS ranking objective is replaced entirely with closed-loop trajectory evaluation. LiDAR-based FITENTH architectures are selected by simulated cross-track error using Bayesian optimization via Optuna [15], making the framework applicable to non-differentiable simulators and straightforward to extend to any closed-loop behavioral metric.

Contributions of this work: The proposed framework defines each NAS candidate $\mathcal{A}_{\text{safe}}$ as a tuple of three independently sampled 1D convolutional networks, one per estimation target.

The search space contains approximately 4×10^{12} valid three-network perception module configurations. Each candidate is trained, inserted into the fixed F1TENTH controller, and ranked by the RMSE of the cross-track error; a 120-trial Bayesian search identifies the best candidate $\mathcal{A}_{\text{safe}}^*$. Final evaluation on six held-out racing tracks uses a normalized composite safety-cost metric aggregating cross-track error, heading-angle error, and wall clearance. $\mathcal{A}_{\text{safe}}^*$ reduces normalized composite safety cost by 27.5% relative to the fixed baseline architectures. Moreover, each individual component improved: 36.8% in cross-track error, 22.8% in heading-angle error, and 24.9% in wall clearance relative to the baseline.

Organization: The remainder of the paper describes the system model and NAS background (Section II), the proposed framework (Section III), experimental setup and results (Section IV), and conclusions (Section V).

II. BACKGROUND

In this section, we begin by describing the F1TENTH platform and its closed-loop system model, covering vehicle dynamics, the LiDAR-based perception and control pipeline, and the trajectory-level metrics used for candidate evaluation. We then review the NAS framework, summarizing conventional and hardware-aware formulations, and discuss how their limitations in cyber-physical settings motivate the safety-informed NAS strategy developed in this work.

A. Closed-Loop F1TENTH System Model

1) *Vehicle Dynamics:* The experiments are performed in the F1TENTH [1] autonomous racing environment, which implements the dynamics specified in [16]. The system state is $x(t) = [x(t), y(t), \delta(t), v(t), \theta(t)]^\top$, where $x(t)$ and $y(t)$ denote global position coordinates, $\delta(t)$ is the steering angle, $v(t)$ is the longitudinal velocity, and $\theta(t)$ is the vehicle heading. The control inputs are steering-rate command $\delta_u(t)$ and longitudinal acceleration $a(t)$. The resulting dynamics are $\dot{x} = v \cos \theta$, $\dot{y} = v \sin \theta$, $\dot{\delta} = \delta_u$, $\dot{v} = a$, $\dot{\theta} = \frac{v}{L} \tan \delta$, where L denotes the wheelbase length. For sampled-data execution with sampling period h , the closed-loop dynamics are written as $x[k+1] = f_h(x[k], u[k])$.

2) LiDAR-Based Perception and Controller Pipeline:

The racing controller operates on data from LiDAR sensing to obtain the vehicle state. At each control step, a one-dimensional LiDAR scan containing 1080 range measurements is processed to estimate three perception-relevant quantities: $y[k] = [d_L[k], w[k], \psi[k]]^\top$, where $d_L[k]$ represents the left-wall distance, $w[k]$ represents track width, and $\psi[k]$ represents heading deviation relative to the track centerline. The training dataset is constructed from input-output pairs where each input is a 1080-beam LiDAR scan and the corresponding labels are the ground-truth left-wall distance, track width, and heading deviation. The three deep neural networks (DNNs) are trained independently using these labels to estimate the vehicle's position and orientation relative to the track centerline.

These quantities are estimated using three neural networks

$$\mathcal{A} = \left(f_{\theta}^{d_L}, f_{\theta}^w, f_{\theta}^{\psi} \right), \quad (1)$$

where each neural network predicts a single geometric quantity from the LiDAR observation. Independent architectures are used instead of a shared multi-task model because the three estimation targets rely on different spatial characteristics of the scan. Estimating wall distance primarily depends on local boundary structure, track-width estimation requires broader spatial coverage, and heading estimation depends strongly on directional asymmetry cues. Allowing each target to use a separate architecture therefore enables the NAS procedure to specialize representational capacity and feature extraction behavior for each estimation task independently.

The estimated geometric quantities are combined with directly measured onboard variables inside the downstream racing controller: $u[k] = \kappa(\tilde{y}[k], \chi[k])$, where $\tilde{y}[k]$ contains the neural-network estimates and $\chi[k]$ denotes auxiliary measured quantities available to the controller. The resulting control commands determine steering and speed behavior during closed-loop execution.

3) *Trajectory-Level Evaluation Metrics:* Rather than ranking neural networks using isolated prediction accuracy, each sampled perception stack (referred to as a candidate architecture) is inserted into the controller pipeline and assessed through the resulting vehicle trajectory. Candidate architectures are therefore evaluated according to the behavior of the complete closed-loop system in simulation.

A reference centerline is defined to exactly bisect the local drivable width. Accordingly, cross-track error (CTE) at each simulation timestep is the distance between the vehicle's position and this centerline. The primary objective used during NAS is the average cross-track root-mean-square error (RMSE):

$$J(\mathcal{A}) = \frac{1}{|\mathcal{T}|} \sum_{\tau \in \mathcal{T}} \text{RMSE}_{\text{CTE}}(\mathcal{A}, \tau), \quad (2)$$

where $\mathcal{T}$ denotes the evaluation-track set and RMSE_{CTE} measures deviation from the desired racing line during rollouts.

B. Neural Architecture Search

Neural Architecture Search (NAS) is an automated framework for identifying effective neural network structures without relying entirely on manual architecture engineering. Rather than selecting model depth, layer width, activation functions, and other architectural components through repeated human tuning, NAS explores a predefined design space to identify architectures that perform well for a target application.

A NAS pipeline typically consists of three main elements: a search space describing the allowable network configurations, a search algorithm that explores candidate architectures, and an evaluation objective used to rank sampled models. Depending on the application, the search space may include architectural choices such as convolutional depth, channel dimensions, pooling structure, optimizer selection, or layer size.

Most existing NAS formulations still evaluate architectures primarily using supervised learning objectives or static efficiency constraints. In cyber-physical systems, however, neural networks operate as components inside larger feedback-control pipelines, where inference delay and estimation error

can jointly influence downstream control behavior. As a result, architectures with similar prediction accuracy or latency characteristics may produce substantially different closed-loop system responses after deployment. Existing performance-efficiency formulations therefore do not fully capture how neural-network design choices affect trajectory stability or system performance in autonomous control applications.

These limitations motivate the use of trajectory-driven evaluation strategies for NAS in autonomous systems. Instead of ranking candidate architectures solely based on prediction accuracy, safety-informed NAS formulations evaluate how DNN outputs influence the entire system during operation.

III. SAFETY INFORMED NAS

In this work, we study whether safety-informed NAS can improve perception architecture selection by replacing handcrafted design and accuracy-based search with safety-informed, closed-loop search. As described in Section II-B, conventional NAS and perception-model design often rank candidate networks using supervised prediction objectives, efficiency metrics, or architectural intuition. Here, each candidate $\mathcal{A}_{\text{safe}}$ is defined as a complete three-network perception module,

$$\mathcal{A}_{\text{safe}} = \left(f_{\theta_{\text{safe}}}^{d_L}, f_{\theta_{\text{safe}}}^w, f_{\theta_{\text{safe}}}^\psi \right), \quad (3)$$

where the three components $f_{\theta_{\text{safe}}}^{d_L}$, $f_{\theta_{\text{safe}}}^w$ and $f_{\theta_{\text{safe}}}^\psi$ correspond to the left-wall distance, track-width, and heading-error networks, respectively. Each candidate is evaluated only after these components are implemented within the downstream Stanley planner [17]. Architectures are then ranked by their performance measured from the simulated vehicle behavior. This distinction is important because standalone prediction accuracy—that is commonly used in practice—cannot fully determine closed-loop safety; estimation errors can propagate differently depending on the predicted quantity, track geometry, and vehicle state, producing differences in cross-track deviation, heading error, steering oscillation, speed stability, and wall-clearance risk. Our proposed safety-informed NAS therefore selects architectures according to trajectory-level consequences.

Figure 1 summarizes the experimental pipeline. Fixed inputs include a 1×1080 LiDAR dataset, the NAS search-space constraints, and the FITENTH simulation environment. These inputs are passed to the NAS loop, where Optuna [15] samples a candidate perception module, trains its three target-specific networks under a restricted search budget, and evaluates the resulting module in closed-loop simulation to obtain RMSE_{CTE} . Optuna is an open-source hyperparameter optimization framework that supports automated, sample-efficient search over complex design spaces using strategies such as Bayesian optimization. This score is returned to the Optuna Tree-structured Parzen Estimator (TPE) [18], and the cycle repeats for n trials. After the search phase is complete, the best candidate is retrained under a longer fixed training profile and compared against the pretrained baseline architectures. Although this implementation uses Optuna’s TPE sampler, the proposed safety-informed NAS framework is not tied to Optuna; any black-box or NAS optimizer that can sample

candidate architectures and update its search from the scalar trajectory-level objective $J(\mathcal{A})$ could be used in its place.

A. Architecture Search Space

For each candidate $\mathcal{A}_{\text{safe}}$, given by (3), the NAS procedure samples three independent target-specific architectures while keeping the controller interface fixed. Each component is built from a sequence of 1D convolutional perception blocks followed by a fully connected prediction head. Each perception block consists of a 1D convolution, an activation function, and a 1D pooling layer. For each component, the search varies: number of convolutional perception blocks, from one to three; activation function within each block, selected from ReLU and ELU; pooling size within each block, selected from $\{2, 3, 4\}$; output-channel width within each block, with the first block selected from $\{1, 4, 8, 16\}$ and later blocks selected from $\{4, 8, 16, 24, 32\}$; fully connected prediction head, selected from 16, 32, 64, 128, 64-32, or 128-64.

Two pruning constraints are used to avoid invalid or uninformative architectures. First, convolutional channel counts must be non-decreasing across successive layers. Second, the flattened representation after convolution and pooling must retain at least 256 features. Architectures that violate either condition are pruned before full evaluation. These constraints prevent the search from allocating trials to models that collapse the LiDAR representation too aggressively or reduce capacity in deeper stages of the network.

The search space is restricted to lightweight convolutional perception models so the experiment tests safety-informed selection and avoids unrestricted search. For a single target network, valid convolutional configurations are counted by depth: $4 \times 3 = 12$ one-layer, $17 \times 3^2 = 153$ two-layer, and $46 \times 3^3 = 1242$ three-layer networks. After the flattened-feature constraint removes 75 three-layer cases, there are 1167 valid three-layer configurations and 1332 valid convolutional configurations overall. Accounting for two activation choices and six prediction-head choices gives $1332 \times 2 \times 6 = 15,984$ architectures per target network. Because each perception module contains three independently sampled target networks, the full space contains $15,984^3 \approx 4.08 \times 10^{12}$ configurations. Because the sensing interface, output targets, and downstream controller are fixed, differences in performance can be attributed to how the candidate perception models are selected.

B. Hidden Layer Blocks

The fixed baseline models and sampled NAS candidates share the same high-level structure: a sequence of convolutional perception blocks followed by a fully connected prediction head. The six baseline architectures (A–F) are drawn from recent literature that evaluated this family of lightweight 1D CNN architectures on the FITENTH platform for the same three estimation tasks [14], providing a well-grounded and platform-validated comparison set. Table I shows this structure for the fixed baseline architectures used in comparison. Each C_o^p block represents one stage of LiDAR feature extraction, where the output-channel count o controls feature capacity

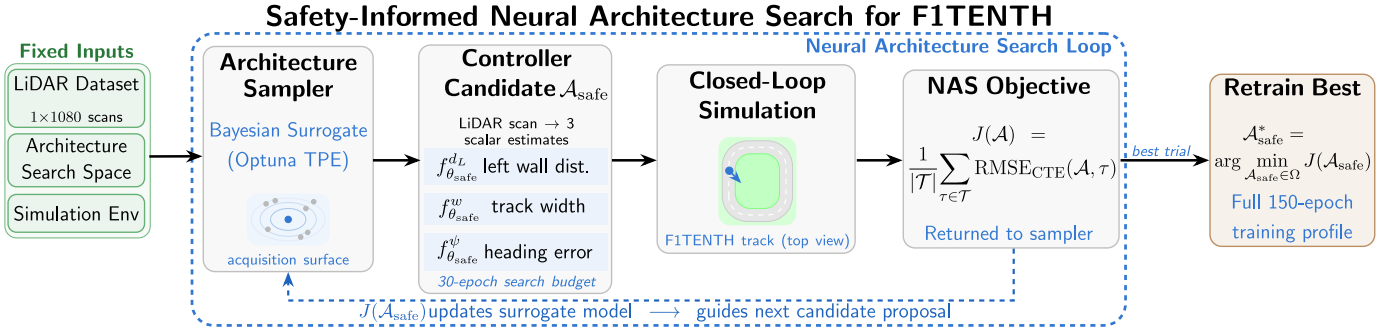


Fig. 1. Overview of the safety-informed NAS pipeline. Each trial samples three target-specific neural networks, trains them under a restricted search budget, evaluates the resulting controller in closed-loop simulation, and returns the trajectory-level objective to the NAS optimizer. The best selected candidate is then retrained and compared against fixed baseline architectures.

and the pooling size p controls spatial downsampling. After the convolutional stack, the flattened representation is mapped through either a single scalar output layer or one hidden fully connected layer before producing the geometric estimate.

TABLE I
FIXED BASELINE CNN ARCHITECTURES

Layer	Architecture					
	A	B	C	D	E	F
Conv. block 1	C_1^8	C_1^4	C_1^2	C_1^2	C_8^2	C_{16}^2
Conv. block 2	–	C_1^2	C_8^4	C_{16}^4	C_{16}^4	C_{32}^4
Conv. block 3	–	C_1^4	–	–	–	–
Flatten dim.	134	32	1072	2144	2144	4288
Hidden FCL	–	8	32	32	64	128
Parameters	417	855	103,215	206,223	413,331	1,652,259

The NAS search generalizes this baseline design pattern instead of introducing a separate model family. Specifically, it preserves the same block-level organization shown in Table I, but allows each target-specific network to vary the number of convolutional blocks, channel widths, pooling sizes, activation function, and fully connected head. This makes the comparison between fixed baselines and NAS-selected candidates architecture-level: both use lightweight 1D CNN perception models, but the NAS candidates are selected by closed-loop trajectory performance.

C. NAS Algorithm

As outlined earlier, the NAS procedure is implemented as an Optuna minimization study over the perception-architecture search space defined above. In each trial, Optuna samples the three target-specific components of a candidate $\mathcal{A}_{\text{safe}}$: one network for left-wall distance, one for track width, and one for heading error. The safety-informed NAS trains these sampled networks on the complete LiDAR dataset, inserted into the fixed downstream F1TENTH controller, and evaluated through closed-loop simulation. The cross-track RMSE is returned to Optuna as the trial objective, allowing later trials to be guided by closed-loop vehicle behavior. Formally, the search selects

$$\mathcal{A}_{\text{safe}}^* = \arg \min_{\mathcal{A}_{\text{safe}} \in \Omega} J(\mathcal{A}_{\text{safe}}), \quad (4)$$

where $J(\mathcal{A}_{\text{safe}})$ is the trajectory-level objective defined in (2) and Ω denotes the valid perception-architecture search space after pruning constraints are applied. Thus, architecture quality is treated as a property of the perception stack after integration

with the fixed F1TENTH controller: a candidate is favored when its three components work together to reduce simulated cross-track error during closed-loop execution.

By contrast, the accuracy-driven NAS follows a traditional procedure. In each iteration, a three-network perception module defined by (1) is sampled, and each component is individually trained on the 80% of the LiDAR dataset before being tested on the remaining 20% for the validation RMSE, given as $\text{RMSE}_{\text{val}}(f_{\theta_{\text{acc}}}^k) = \sqrt{\frac{1}{N} \sum_{j=1}^N (f_{\theta_{\text{acc}}}^k(x_j) - y_j^k)^2}$, where $k \in \{d_L, w, \psi\}$, N is the number of validation samples for a given perception component, $f_{\theta_{\text{acc}}}^k$ is the prediction made by the component for sample j , and y_j^k is the corresponding ground-truth value. This target is used by Optuna for each NAS cycle:

$$\mathcal{A}_{\text{acc}}^* = \arg \min_{\mathcal{A}_{\text{acc}} \in \Omega} \text{RMSE}_{\text{val}}(f_{\theta_{\text{acc}}}^k), \quad (5)$$

where Ω is as previously defined for the safety-informed NAS. To clarify the training for the accuracy-driven NAS, each perception module component is trained independently for its specific task using three separate Optuna studies. For each individual network, the evaluation workflow calculates raw prediction errors on the validation dataset, squares and averages them to obtain the Mean Squared Error (MSE), and takes the square root to compute the validation RMSE, which Optuna then minimizes. Because each target network is optimized through its own dedicated study, no combination or cross-metric weighting of the three validation RMSE values is required during architecture selection.

IV. EXPERIMENTS

Our evaluation tests whether NAS-selected perception improves closed-loop driving behavior relative to handcrafted baseline architectures. The central comparison assesses safety in simulation rather than isolated neural network validation losses. Each perception module receives the same 1080-beam LiDAR input, predicts the same three geometric quantities, and is evaluated using the same metrics. This design isolates the effect of architecture selection on closed-loop behavior.

A. Setup

Each safety-informed NAS run selects $\mathcal{A}_{\text{safe}}^*$ through a 120-trial search as given by (4). The accuracy-driven NAS run selects $\mathcal{A}_{\text{acc}}^*$ given by the standard NAS formulation defined in (5). Although 120 trials cover only a small fraction of the total

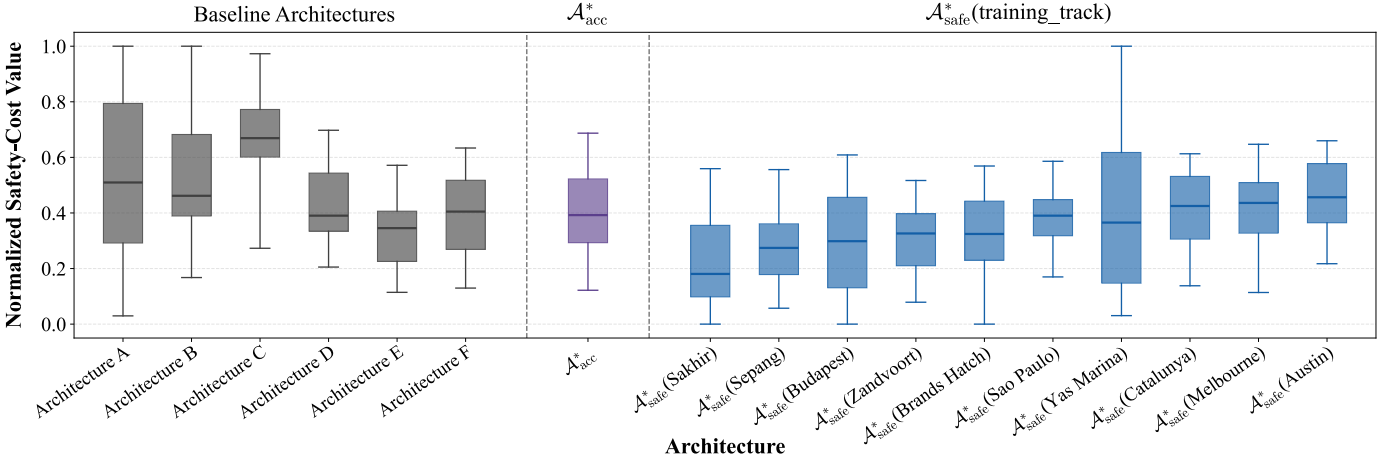


Fig. 2. Composite safety-cost distribution by architecture group. The accuracy-driven NAS candidate (purple) has lower composite cost relative to the handcrafted baselines (gray). Safety-informed NAS candidates (blue) show the lowest composite cost. Lower values indicate safer closed-loop behavior.

search space, Optuna’s TPE surrogate concentrates evaluations in architecturally promising regions, making surrogate-guided search practical at budgets far smaller than exhaustive enumeration would require. The complete LiDAR dataset was first partitioned using a fixed random seed of 42 into an 80% development set and a 20% held-out architecture-evaluation set. Within each NAS trial, 10% of the development set was reserved for internal validation and early stopping, resulting in effective proportions of 72% for model fitting, 8% for internal validation, and 20% for held-out architecture evaluation. During the search phase, each trial trains three target-specific networks, shown by (3), under a restricted 30-epoch budget using batch size 128, an internal validation split of 0.1, learning rate 5×10^{-5} , weight decay 10^{-4} , gradient clipping of 1.0, early stopping patience of 15 epochs, a fixed random seed of 42, and a ReduceLROnPlateau scheduler with factor 0.5 and patience 20. Notably, the three perception networks are also trained simultaneously, reducing the training time of each NAS trial to the maximum runtime among the three heads. The trained perception module is then inserted into the fixed F1TENTH controller and ranked by closed-loop cross-track RMSE. After the search completes, the best candidate from each run is retrained under a fixed 150-epoch profile before final evaluation. The final comparison uses the retrained NAS-selected $\mathcal{A}_{\text{safe}}^*$ candidates and the fixed pretrained baselines. Each candidate is evaluated against the baseline family on six held-out test tracks (taken from [19]): Mexico City, Monza, Nürburgring, Silverstone, Sochi, and Spa. For each run, the simulator records max cross-track error, max heading angle error, and mean wall clearance. These logged quantities capture different aspects of closed-loop behavior. Max cross-track error (CTE) measures the greatest deviation of the F1TENTH car from the desired path. Max heading error (heading) reflects orientation error and provides insight into the reactivity of the system. Raw clearance is a higher-is-better metric and is therefore converted into the lower-is-better clearance risk $\tilde{m}_{\text{clear}}(\mathcal{A}) = 1 - \frac{m_{\text{clear}}(\mathcal{A}) - m_{\text{clear}}^{\min}}{m_{\text{clear}}^{\max} - m_{\text{clear}}^{\min}}$, where $m_{\text{clear}}(\mathcal{A})$ is the minimum LiDAR range in the trajectory.

These behaviors are summarized into a single score where

we derive a normalized composite safety-cost metric computed as the mean of the normalized cost terms: $C(\mathcal{A}) = \frac{1}{K} \sum_{k=1}^K \tilde{m}_k(\mathcal{A})$, where K is the number of metrics included in the composite and $\tilde{m}_k(\mathcal{A})$ is the normalized value of metric k . For each metric, normalization is computed as $\tilde{m}_k(\mathcal{A}) = \frac{m_k(\mathcal{A}) - m_k^{\min}}{m_k^{\max} - m_k^{\min}}$, where $m_k(\mathcal{A})$ is the aggregate unnormalized value of metric k for architecture $\mathcal{A}$, and $m_k^{\min}$ and $m_k^{\max}$ are the minimum and maximum aggregate values of that metric across the compared architectures. The composite score is used as the paper’s operational safety measure. By aggregating the normalized trajectory-level costs into a single lower-is-better value, it provides a consistent basis for comparing how safely each perception architecture behaves after integration with the fixed F1TENTH controller. We therefore report improvements as reductions in composite safety cost.

B. Results

Figure 2 shows the distribution of normalized composite safety-cost values for the NAS-selected perception modules and the fixed baseline architectures. The purple accuracy-driven NAS candidate ($\mathcal{A}_{\text{acc}}^*$) shows an improvement against the gray baseline values. Meanwhile, the blue safety-informed NAS candidates ($\mathcal{A}_{\text{safe}}^*$) shift the distribution toward lower composite safety cost relative to both $\mathcal{A}_{\text{acc}}^*$ and the pooled baseline family, indicating improved closed-loop behavior in conjunction with the F1TENTH controller. To separate the effect of the closed-loop objective from the effect of NAS itself, we additionally compare against an accuracy-driven NAS configuration ($\mathcal{A}_{\text{acc}}^*$) in which architectures are selected based on prediction accuracy rather than trajectory-level evaluation.

Table II summarizes the component-wise normalized safety-cost results. Both NAS approaches outperform the pooled baseline mean across all reported components, showing that architecture search alone improves the perception stack relative to the fixed manually designed architectures. However, the safety-informed search consistently produces the largest improvements. Relative to the baseline mean, while $\mathcal{A}_{\text{acc}}^*$ reduces normalized CTE, heading, and clearance costs, $\mathcal{A}_{\text{safe}}^*$ achieves much larger reductions of 36.8%, 22.8%, and 24.9%.

TABLE II
COMPONENT-WISE NORMALIZED SAFETY COSTS BY ARCHITECTURE. $\Delta(\cdot)$
DENOTES THE PERCENTAGE REDUCTION RELATIVE TO THE BASELINE.

Architecture	CTE		Heading		Clearance Risk	
	Mean	Std	Mean	Std	Mean	Std
Baseline A	0.476	0.341	0.583	0.287	0.509	0.343
Baseline B	0.459	0.270	0.666	0.201	0.517	0.257
Baseline C	0.673	0.175	0.649	0.155	0.666	0.244
Baseline D	0.336	0.084	0.536	0.165	0.419	0.145
Baseline E	0.262	0.089	0.405	0.189	0.351	0.124
Baseline F	0.311	0.108	0.464	0.169	0.404	0.158
Baseline Mean	0.420	0.233	0.551	0.208	0.478	0.233
$\mathcal{A}_{acc}^*$ Mean	0.321	0.093	0.437	0.241	0.402	0.130
$\mathcal{A}_{safe}^*$ Mean	0.265	0.153	0.425	0.189	0.359	0.160
$\Delta(\mathcal{A}_{acc}^*)$	23.6%	60.0%	20.6%	-15.9%	15.8%	44.1%
$\Delta(\mathcal{A}_{safe}^*)$	36.8%	34.4%	22.8%	9.0%	24.9%	31.4%

C. Analysis

The comparison between $\mathcal{A}_{acc}^*$ and $\mathcal{A}_{safe}^*$ shows that prediction accuracy alone is insufficient to characterize deployed perception quality. The three learned perception outputs are coupled by the FITENTH controller, allowing small estimation errors to propagate into steering oscillations, reduced wall-clearance, or unstable trajectory corrections during closed-loop simulation. Consequently, architectures that have higher accuracy may still yield less safe behavior. In contrast, the safety-informed NAS objective captures these interactions directly by evaluating simulated trajectories instead of perception errors.

The component-wise results also motivate the use of a composite safety metric. No single baseline architecture consistently dominates across all behaviors, and architecture rankings vary depending on whether the emphasis is trajectory tracking, heading stability, or wall-clearance behavior. Optimizing solely for cross-track RMSE during the NAS search phase serves as a highly effective proxy for overall system behavior, avoiding the complexity of balancing multiple conflicting objectives during optimization. Crucially, the final evaluation reveals a positive transfer effect: training the perception modules to minimize cross-track error implicitly teaches them features that benefit overall vehicle stability. As a result, the safety-informed architectures naturally achieve superior performance in metrics not explicitly optimized for like heading error and wall clearance, validating that cross-track error optimization acts as a strong driver for holistic closed-loop safety. Under this trajectory-level evaluation, $\mathcal{A}_{safe}^*$ achieves the most consistent overall tradeoff across measured behaviors.

The proposed framework optimizes trajectory-level metrics in simulation rather than explicit constraints like collision or lap completion rates. However, unsafe behaviors like collisions are naturally penalized during the search phase because an interrupted or failing rollout yields an exceptionally high cross-track RMSE, driving the Optuna optimizer away from those regions of the search space. Across all evaluations on the six held-out test tracks, the final selected architectures achieved full lap completions without collisions.

One limitation of the normalization procedure is that the reported reductions are relative to the evaluated architecture

set. Since each metric is normalized using the minimum and maximum values observed across compared architectures, the absolute normalized values would change if additional architectures were introduced. The reported improvements should therefore be interpreted within the evaluated perception-module family rather than as absolute safety guarantees.

V. CONCLUDING REMARKS AND FUTURE WORK

This work highlights the benefits of incorporating *system-level* safety objectives into neural architecture search for autonomous control systems. It presents a safety-informed NAS framework for selecting LiDAR-based perception architectures in a closed-loop FITENTH autonomous racing environment. Rather than ranking candidate architectures using only supervised prediction loss, each candidate perception module is trained, embedded into a fixed FITENTH controller, and evaluated according to the resulting simulated driving behavior through cross-track error. The best candidate identified through the 120-trial Bayesian search is then retrained and compared against both fixed handcrafted baseline architectures and an accuracy-driven NAS configuration across six held-out tracks.

The experimental results show that both NAS-based approaches outperform the handcrafted baseline family, demonstrating that automated architecture selection improves the perception stack relative to manually designed architectures. However, the safety-informed NAS configuration $\mathcal{A}_{safe}^*$ consistently achieves the strongest closed-loop performance, reducing normalized cross-track error, heading error, and clearance cost by 36.8%, 22.8%, and 24.9% relative to the baseline mean, compared to 23.6%, 20.6%, and 15.8% for the accuracy-driven $\mathcal{A}_{acc}^*$. The largest gain is in cross-track error, which is particularly important because cross-track deviation is tied to unstable corrective steering and large trajectory departures during driving.

The results support the paper’s central claim that perception architectures used with feedback-control systems should be selected using trajectory-level evaluation rather than isolated prediction accuracy alone. Although the accuracy-driven NAS improves prediction quality sufficiently to outperform the fixed handcrafted baselines, directly optimizing for closed-loop vehicle behavior produces architectures with consistently better trajectory tracking and safety metrics after deployment. The proposed framework remains safety-informed rather than formally safety-certified. It does not impose hard safety constraints or provide formal guarantees on collision avoidance, stability, or lap completion, and the reported normalized improvements are relative to the evaluated architecture family, since the composite safety-cost metric is normalized within the comparison set.

Future Work: Extensions should explore optimizing the composite safety cost directly during search, evaluating candidates across multiple training tracks, extending the search to learned planners or controllers, and validating on physical hardware. Additionally, training should be performed on procedurally-generated tracks due to the fixed tracks lacking variability of diverse features.

REFERENCES

- [1] M. O’Kelly, H. Zheng, D. Karthik, and R. Mangharam, “FITENTH: An Open-source Evaluation Environment for Continuous Control and Reinforcement Learning,” in *Proceedings of Machine Learning Research (PMLR)*, 2019, pp. 77–89.
- [2] T. Elsken, J. H. Metzen, and F. Hutter, “Neural Architecture Search: A Survey,” *Journal of Machine Learning Research*, vol. 20, pp. 55:1–55:21, 2019.
- [3] B. Zoph and Q. V. Le, “Neural Architecture Search with Reinforcement Learning,” in *International Conference on Learning Representations (ICLR)*, 2017.
- [4] E. Real, S. Moore, A. Selle, S. Saxena, Y. L. Suematsu, J. Tan, Q. Le, and A. Kurakin, “Large-Scale Evolution of Image Classifiers,” in *International Conference on Machine Learning (ICML)*, 2017, pp. 2902–2911.
- [5] H. Liu, K. Simonyan, and Y. Yang, “DARTS: Differentiable Architecture Search,” in *International Conference on Learning Representations (ICLR)*, 2019.
- [6] H. Cai, L. Zhu, and S. Han, “ProxylessNAS: Direct Neural Architecture Search on Target Task and Hardware,” in *International Conference on Learning Representations (ICLR)*, 2019.
- [7] K. T. Chitty-Venkata and A. K. Somani, “Neural Architecture Search Survey: A Hardware Perspective,” *ACM Computing Surveys (CSUR)*, vol. 55, no. 4, pp. 78:1–78:36, 2023.
- [8] Q. Lu, W. Jiang, X. Xu, Y. Shi, and J. Hu, “On Neural Architecture Search for Resource-Constrained Hardware Platforms,” *arXiv*, 2019. [Online]. Available: <http://arxiv.org/abs/1911.00105>
- [9] H.-D. Tran, W. Xiang, and T. T. Johnson, “Verification Approaches for Learning-Enabled Autonomous Cyber-Physical Systems,” *IEEE Magazine of Design & Test (MDAT)*, vol. 39, no. 1, pp. 24–34, 2022.
- [10] C. Hobbs, B. Ghosh, S. Xu, P. S. Duggirala, and S. Chakraborty, “Safety Analysis of Embedded Controllers Under Implementation Platform Timing Uncertainties,” *IEEE Transactions on Computer Aided Design Integrated Circuits Systems (TCAD)*, vol. 41, no. 11, pp. 4016–4027, 2022.
- [11] P. Ganguli, T. Zhu, and S. Chakraborty, “Scheduling Control Tasks Using Safety-Guided RL,” in *International Conference on Applied Algorithms (ICAA)*, 2026, pp. 169–182.
- [12] S. Xu, C. Hobbs, Y. Song, B. Ghosh, T. Zhu, S. Aktar, L. Yang, Y. Sheng, W. Jiang, J. Hu, P. S. Duggirala, and S. Chakraborty, “GPU Partitioning & Neural Architecture Sizing for Safety-Driven Sensing in Autonomous Systems,” in *International Conference on Assured Autonomy (ICAA)*, 2024, pp. 67–76.
- [13] K. Samal, M. Wolf, and S. Mukhopadhyay, “Closed-loop Approach to Perception in Autonomous System,” in *Design, Automation & Test in Europe Conference & Exhibition (DATE)*, 2021, pp. 463–468.
- [14] T. Zhu, M. Li, B. Ghosh, S. Chakraborty, and P. S. Duggirala, “Safety-Driven DNN Sizing for Vehicular CPS,” *IEEE Embedded Systems Letters (ESL)*, vol. 18, no. 2, pp. 164–167, 2026.
- [15] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, “Optuna: A Next-generation Hyperparameter Optimization Framework,” in *International Conference on Knowledge Discovery & Data Mining (KDD)*, 2019, pp. 2623–2631.
- [16] M. Althoff, M. Koschi, and S. Manzing, “Commonroad: Composible benchmarks for motion planning on roads,” in *Intelligent Vehicles Symposium (IV)*, 2017.
- [17] G. M. Hoffmann, C. J. Tomlin, M. Montemerlo, and S. Thrun, “Autonomous automobile trajectory tracking for off-road driving: Controller design, experimental validation and racing,” in *American Control Conference (ACC)*, 2007.
- [18] J. Bergstra, R. Bardenet, Y. Bengio, and B. Kégl, “Algorithms for hyper-parameter optimization,” in *International Conference on Neural Information Processing Systems (NeurIPS)*, 2011.
- [19] J. Betz, H. Zheng, A. Liniger, U. Rosolia, P. Karle, M. Behl, V. Krovi, and R. Mangharam, “Autonomous vehicles on the edge: A survey on autonomous vehicle racing,” *Open Journal of Intelligent Transportation Systems (OJITS)*, vol. 3, pp. 458–488, 2022.